# 🌐 Module 06: API and Java Web Token Penetration Testing

```
# Exercise 09: Exploiting XPath Injection Vulnerabilities in API Implementations
```

### Scenario

XPath Injection occurs when web applications construct XML Path Language (XPath) queries using unvalidated user input to query XML data stores or API configuration files. When inputs passed to API URI parameters (such as `/api/v2/release/<version>`) are concatenated into XPath query strings without proper sanitization or parameterized querying, attackers can inject boolean logic expressions to alter the query logic.

In **DVWS-Node**, the application queries XML release documentation via the endpoint `/api/v2/release/0.0.1`. Injecting a boolean payload such as **0.0.1' or 1=1 or 'a'='a** forces the underlying XML parser to evaluate the condition to `true` for all nodes in the document. Consequently, the API response dumps complete XML configuration contents, exposing sensitive user credentials, hash values, and administrative account data.

In this exercise, we access DVWS on Parrot Security, intercept API release requests in Burp Suite, construct an XPath injection payload (**0.0.1' or 1=1 or 'a'='a**), replay the request in Repeater, and extract sensitive system credentials from the XML response.

### Objective

The objective of this lab is to help students learn how to:
- Identify XML-backed API endpoints vulnerable to XPath Injection (**/api/v2/release/0.0.1**)
- Intercept and inspect API request parameters in Burp Suite Proxy HTTP history
- Construct boolean-based XPath injection payloads (**0.0.1' or 1=1 or 'a'='a**)
- Replay and execute modified HTTP GET requests using Burp Repeater
- Extract sensitive configuration data and user credentials from dumped XML API responses

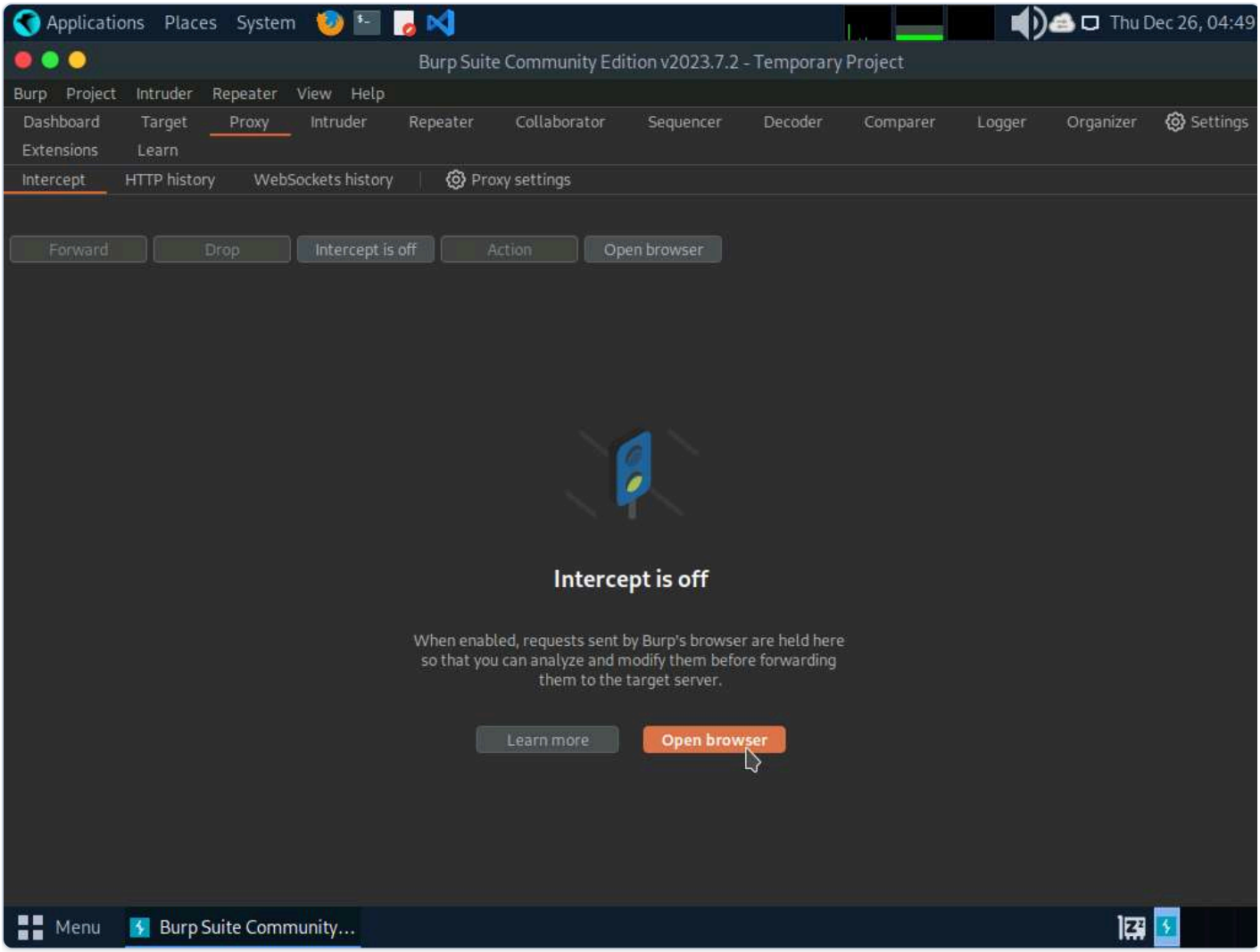
---

## Task 1: Exploit XPath Injection in DVWS API Endpoints

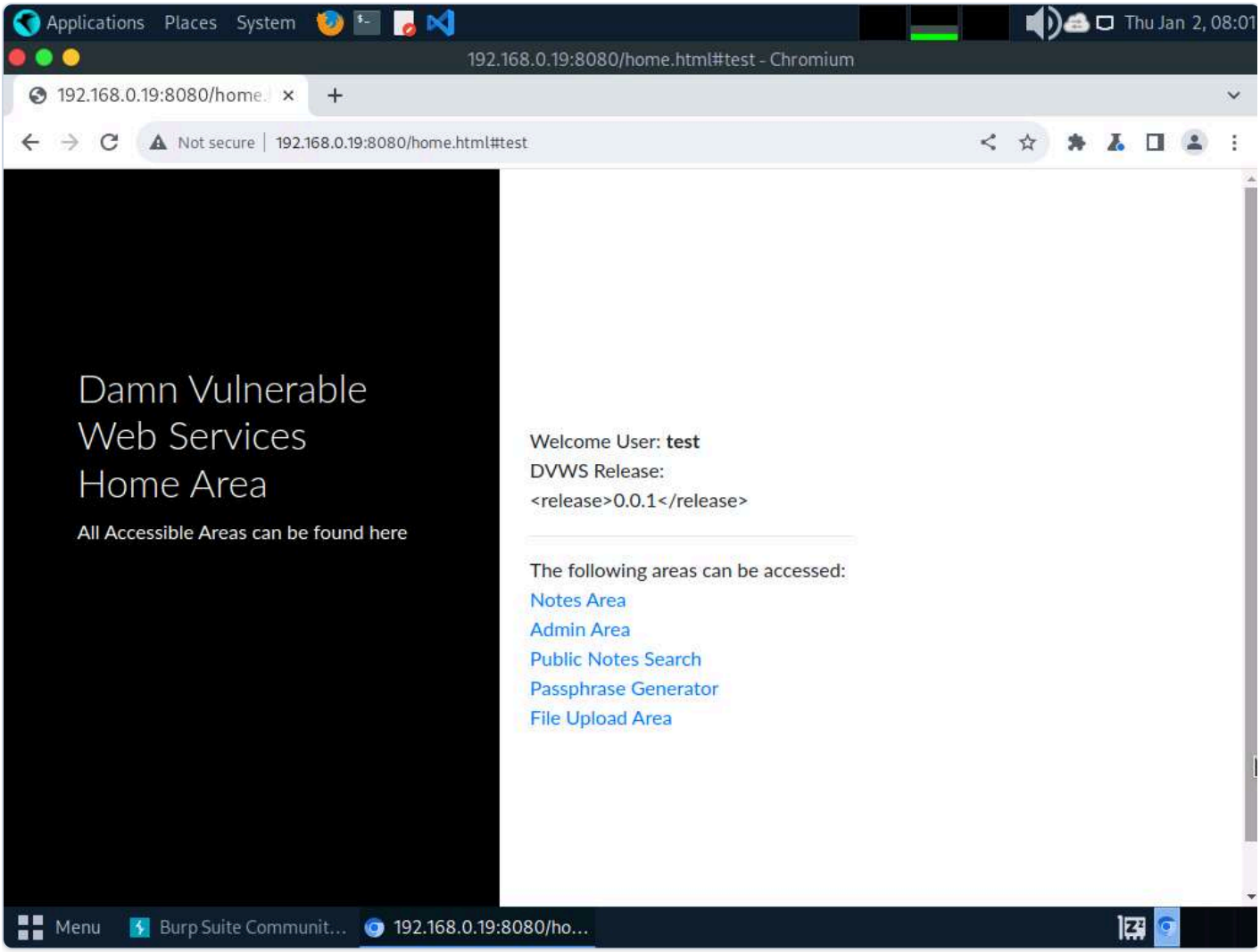
In this task, we log into DVWS on Parrot Security, locate API release GET requests in Burp Suite HTTP history, append an XPath injection boolean payload in Repeater, and extract user credentials from the XML response.

### Lab Tasks

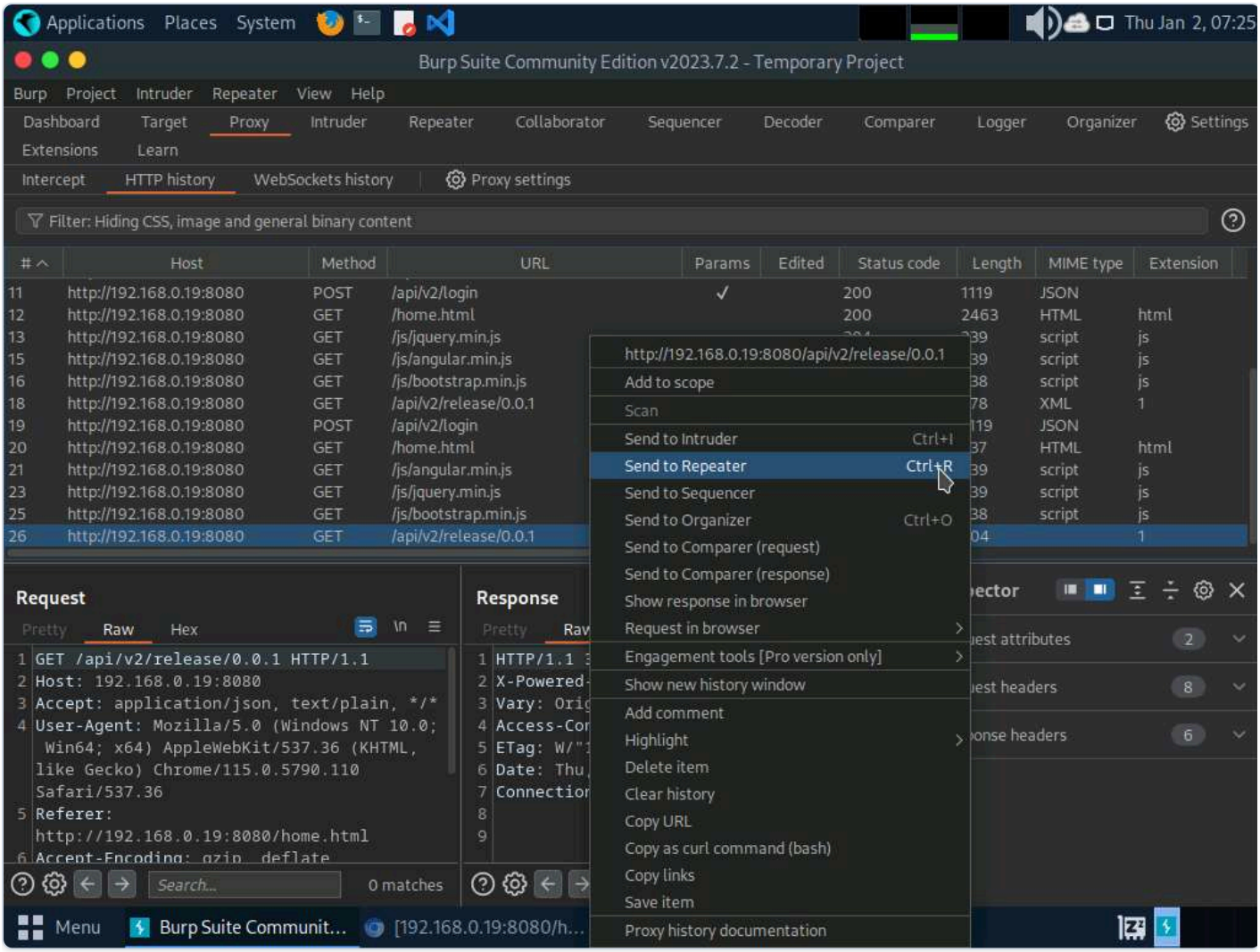
1. Switch to **Parrot Security** (**attacker** / **toor**).
2. Launch **Burp Suite Community Edition**, navigate to **Proxy** tab, set **Intercept is OFF**, and click **Open browser**.



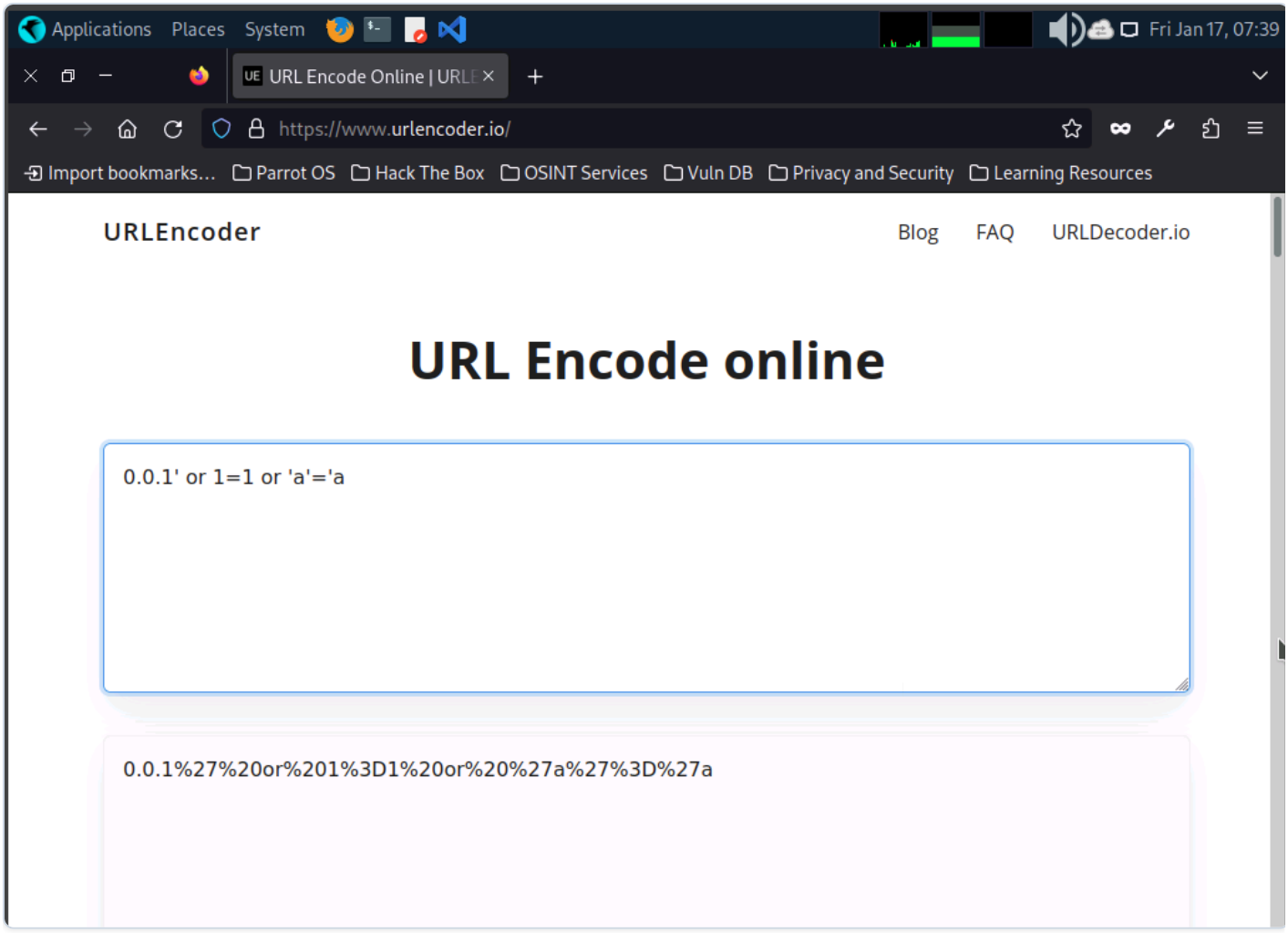

1. In Burp's browser, navigate to **http://192.168.0.19:8080**.
2. If you have not created an account, enter a username and password, click **Register**, and then click **Login**.
3. Switch to Burp Suite and click **Forward** to process all login HTTP transactions.



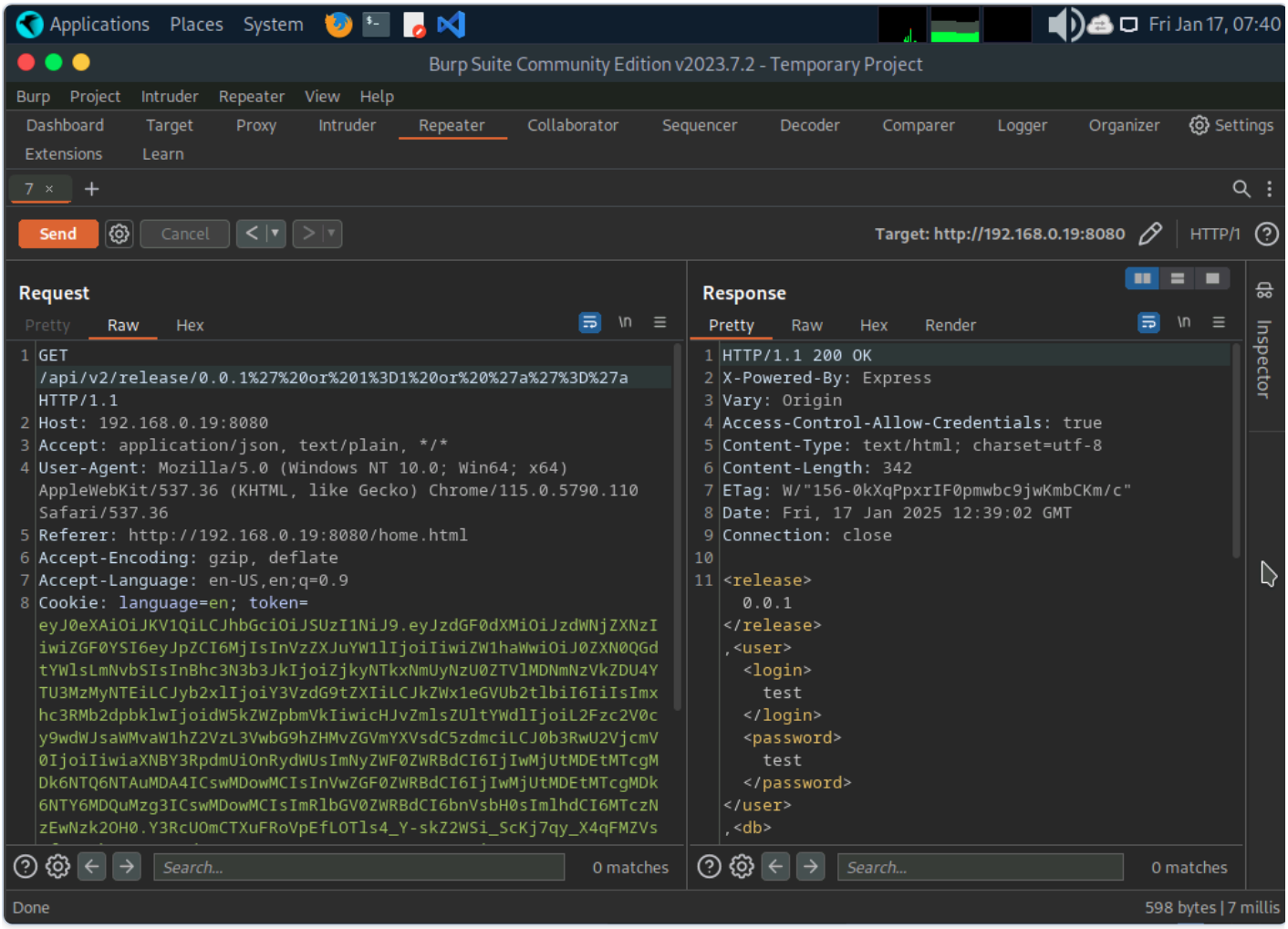

1. Once logged in, open the **Proxy** $ ightarrow$ **HTTP history** tab in Burp Suite.
2. Locate the GET request targeting **/api/v2/release/0.0.1**, right-click, and select **Send to Repeater**.




1. In the **Repeater** tab, modify the URI path parameter to append the boolean XPath injection payload:
   **0.0.1' or 1=1 or 'a'='a**




1. Click **Send** to issue the modified request.




1. Inspect the HTTP 200 OK response to confirm successful extraction of system configuration contents and user credentials (including **root** credentials).
2. Close all browser and Burp Suite windows.

This concludes the lab exercise.

---

**IMPORTANT:** ❓ **Question 6.9.1.1** Utilize Burp Suite within Parrot Security to exploit XPath Injection vulnerabilities in DVWS hosted on an Ubuntu machine using an encoded payload. Input the encoded payload as demonstrated in this lab. [!SUCCESS] ✅ **Answer:** **0.0.1' or 1=1 or 'a'='a**